

Instructions: 42 marks | 70 min | **Set A** | MCQs: $11 \times 3 = 33$ marks (one correct, -1 wrong) | Subjective: 9 marks (6+3)

Quick Reference

Cross-Validation:

`cross_val_score(m, X, y, cv=5)`

Report: mean $\pm$ std

Tuning:

`GridSearchCV(m, params, cv=5)`

`RandomizedSearchCV(n_iter=)`

1. (3 points) [MCQ] What is the relationship between a Docker **image** and a Docker **container**?

- A. A container must be created first before an image can be built from it
- B. They are the same thing, just with different names used in different contexts
- C. An image runs on the cloud; a container runs on your local machine only
- D. An image is a read-only blueprint; a container is a running instance of it**
- E. An image contains only Python code; a container contains only data files
- F. An image is a virtual machine; a container is a lightweight process inside it
- G. An image is a running process; a container is a file stored on disk
- H. A container is always larger in disk size than the image it came from

Solution

Image = recipe (read-only, stored on disk). Container = dish being cooked (running instance with a writable layer on top). Multiple containers can run from the same image simultaneously.

Why “container is always larger” is wrong: a container adds only a thin writable layer on top of the image. If the container writes no files, it is essentially the same size. The image layers are *shared* across containers, not duplicated.

Answer: **Image = read-only blueprint, Container = running instance**

2. (3 points) [MCQ] In Bayesian hyperparameter optimization, what are the two components that guide the search?

- A. A surrogate model and an acquisition function**
- B. A random number generator paired with a fixed stopping criterion after N iterations
- C. A grid of candidate values and an exhaustive evaluator that tries every combination
- D. A neural network mapping hyperparameters to scores with a softmax output
- E. A decision tree splitting on hyperparameter ranges and a pruning strategy
- F. A compiler that optimizes model code and a runtime profiler for latency
- G. A training set for fitting the model and a validation set for scoring candidates
- H. A gradient descent optimizer and a cross-entropy loss applied to hyperparameters

Solution

The **surrogate model** (e.g., Gaussian Process) predicts performance and uncertainty for untried values. The **acquisition function** (e.g., Expected Improvement) balances exploration vs. exploitation. Answer: **Surrogate model + acquisition function**

3. (3 points) [MCQ] You tune a model with 5 hyperparameters. Each has 10 candidate values. How many total combinations does `GridSearchCV` evaluate, and why is `RandomizedSearchCV` often preferred in this setting?

- A. 10^5 ; but random search always finds the true global optimum in fewer iterations
- B. $10^5 = 100,000$; random search samples a fixed budget, exploring more unique values**
- C. $5 \times 10 = 50$; random search adds Gaussian noise to each grid point it evaluates
- D. 50; random search is actually slower but produces a more thorough final result
- E. 500; random search uses Bayesian optimization internally to guide the search
- F. 50; random search skips unpromising combinations automatically using pruning
- G. 100; grid search is faster because it parallelizes across all hyperparameters
- H. 10; grid search evaluates one hyperparameter at a time independently of others

Solution

Grid: $10^5 = 100,000$ combinations. With 5-fold CV = 500,000 fits. Random search samples a budget (e.g., `n_iter=100`) from continuous distributions, exploring more unique values per important parameter. Answer: 10^5 ; **random samples a fixed budget**

4. (3 points) [MCQ] In Bayesian optimization, the acquisition function scores a candidate. A point with **high predicted performance** and **high uncertainty** will get:
- A. An infinite score — high uncertainty causes division by zero in the formula
 - B. A negative score — the acquisition function penalizes any uncertainty as risk
 - C. A score determined solely by the learning rate of the surrogate model
 - D. The same score as a low-mean, low-uncertainty point, since they balance out
 - E. A low score — uncertainty means the surrogate prediction cannot be trusted at all
 - F. A score depending only on predicted performance; uncertainty has no effect
 - G. A high score — promising mean plus large upside from exploration**
 - H. Exactly zero — the exploration and exploitation terms always cancel out

Solution

Acquisition functions (e.g., $UCB = \mu + \beta\sigma$) reward both exploitation (high μ) and exploration (high σ). A point that is promising *and* uncertain gets a high score. Answer: **High score — promising + exploration upside**

5. (3 points) [MCQ] Two annotators label 100 items on a 3-point scale (1, 2, 3). The table below shows how many items each annotator-pair combination received. For example, the cell (row=1, col=2) means Annotator B said “1” and Annotator A said “2” for 5 items.

B ↓ A →	1	2	3	Total
1	30	5	0	35
2	5	20	5	30
3	0	5	30	35
Total	35	30	35	100

Compute P_o (observed agreement = sum of diagonal / N), then P_e (chance agreement = $\sum_i p_i \cdot q_i$ where p_i are row-total proportions and q_i are column-total proportions), and finally κ . What is Cohen's κ (to two decimal places)?

- A. 0.67
- B. 0.60
- C. 0.55
- D. 0.80
- E. 0.70**
- F. 0.65
- G. 0.75
- H. 0.73

Solution

$$P_o = (30 + 20 + 30)/100 = 0.80.$$

Row marginals (B): $p_1 = 0.35$, $p_2 = 0.30$, $p_3 = 0.35$. Column marginals (A): $q_1 = 0.35$, $q_2 = 0.30$, $q_3 = 0.35$.

$$P_e = (0.35)(0.35) + (0.30)(0.30) + (0.35)(0.35) = 0.1225 + 0.09 + 0.1225 = 0.335.$$

$$\kappa = \frac{0.80 - 0.335}{1 - 0.335} = \frac{0.465}{0.665} = 0.699 \approx \mathbf{0.70}.$$

6. (3 points) [MCQ] You have chest X-rays from 200 patients, with 5 images per patient (1000 total). You use standard `KFold(n_splits=5)`. What is the risk?
- A. There is no risk; standard `KFold` handles grouped data correctly by default

- B. KFold automatically keeps all images of a patient together in one fold
- C. The model will train on only 1 patient per fold, causing extreme underfitting
- D. KFold cannot process image data; it requires tabular numerical input
- E. The model will memorize patient IDs as features and fail on new patients
- F. The model will only learn from 40 patients per fold, which is too few
- G. The folds will be too small because 5 images per patient is insufficient
- H. Same-patient images may land in both train and validation, inflating scores**

Solution

If patient A's images split across train/val, the model leaks patient-specific patterns. Fix: `GroupKFold(groups=patient_id)`. Answer: **Same-patient images in both splits = leakage**

7. (3 points) [MCQ] You train a `RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)`. After `model.fit(X_train, y_train)`, which is a **learned parameter**?
- A. `random_state=42`
 - B. The number of samples in `X_train` used for fitting the model
 - C. The `test_size` passed to `train_test_split` earlier in the pipeline
 - D. `max_depth=10`
 - E. The value of `cv` that would be used if you ran cross-validation
 - F. `n_estimators=100`
 - G. The number of features (columns) present in `X_train`
 - H. The split thresholds and feature indices chosen at each node during training**

Solution

Hyperparameters (`n_estimators`, `max_depth`, `random_state`) are set *before* training. Learned parameters (split thresholds, feature selections) are determined *during fit()*. Answer: **Split thresholds and feature indices**

8. (3 points) [MCQ] Consider these two Dockerfiles. You change **only** `app.py` and rebuild. Which rebuilds faster?

Version A:

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 EXPOSE 7860
6 CMD ["python", "app.py"]
```

Version B:

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install -r requirements.txt
5 COPY . .
6 EXPOSE 7860
7 CMD ["python", "app.py"]
```

- A. Neither — Docker does not cache layers between builds by default
- B. B — but only on Linux hosts, not on macOS or Windows with Docker Desktop
- C. B — `app.py` change only invalidates the last COPY; `pip install` stays cached**
- D. Both identical — Docker always rebuilds every layer from scratch on any change
- E. B — but only when using `python:3.10-alpine` as the base image
- F. A — fewer lines means fewer layers, which means a faster rebuild overall
- G. A — Docker caches the `RUN` line regardless of what changed above it
- H. A — a single `COPY . .` is faster than two separate `COPY` commands

Solution

Docker caches each layer. A changed layer invalidates *all subsequent layers*. In A, changing `app.py` invalidates `COPY . .` → forces `pip install` to re-run. In B, `requirements.txt` is unchanged → `pip install` cached → only final `COPY . .` rebuilt. Answer: **Version B**

9. (3 points) [MCQ] Your dataset has 1000 samples: 900 class A and 100 class B. You use standard `KFold(n_splits=5, shuffle=True)`. What is the **most likely problem**?

- A. Cross-validation takes $5\times$ longer than a single split, making it impractical
- B. The model will always predict class A regardless of input features
- C. KFold deterministically puts all class B samples into one fold
- D. Shuffling makes results non-reproducible even when `random_state` is set
- E. Some folds may have very few class B samples, producing unreliable scores**
- F. KFold requires the dataset to be sorted by class label before splitting
- G. Class imbalance only matters for regression, not for classification tasks
- H. KFold cannot handle more than 2 classes in the label column

Solution

With 90/10 imbalance, random folds may get 0–2% class B while others get 15–20%. Fix: `StratifiedKFold` ensures every fold has the same class ratio (90/10). Answer: **Some folds may have very few class B samples**

10. (3 points) [MCQ] You are predicting tomorrow's stock price using 3 years of daily price data (one value per day). Which cross-validation strategy is correct?
- A. `KFold(n_splits=5, shuffle=True)`
 - B. Train on all 3 years, evaluate on the same 3 years
 - C. `TimeSeriesSplit(n_splits=5)`**
 - D. `KFold(n_splits=5, shuffle=False)`
 - E. `StratifiedKFold(n_splits=5)`
 - F. A single 80/20 random split
 - G. `GroupKFold` with `groups = stock ticker symbols`
 - H. Leave-One-Out CV

Solution

Shuffling time series leaks future into training. `TimeSeriesSplit` ensures training uses only *past* data, validation uses *future*. `KFold(shuffle=False)` still mixes past/future across folds. Answer: **TimeSeriesSplit**

11. (3 points) [MCQ] You built `my-app` and ran `docker run -d my-app`. To check it is **still running**, which command lists **currently running containers**?
- A. `docker images`
 - B. `docker pull my-app`
 - C. `docker ps`**
 - D. `docker build .`
 - E. `docker logs my-app`
 - F. `docker system prune`
 - G. `docker stop my-app`
 - H. `docker run my-app`

Solution

`docker ps` lists running containers. Add `-a` for stopped containers too. `docker images` lists images (blueprints), not containers. `docker run` starts a *new* container. Answer: **docker ps**

12. (6 points) [Subjective — Bayesian Optimization] You are tuning `learning_rate` for a gradient boosting model using Bayesian optimization. After 4 evaluations, the Gaussian Process surrogate predicts μ (expected accuracy) and σ (uncertainty) for 3 untried candidates P, Q, R:

	learning_rate	Accuracy	μ	σ
<i>Observed:</i>	0.01	0.78		
	0.1	0.84 (best so far, f^*)		
	0.5	0.80		
	0.9	0.70		
P	0.05	?	0.81	0.02
Q	0.2	?	0.83	0.05
R	0.7	?	0.75	0.08

- (a) (1 mark) Compute the **Upper Confidence Bound** $UCB(x) = \mu(x) + \beta \cdot \sigma(x)$ with $\beta = 2$ for P, Q, R. Which candidate does UCB select next?
- (b) (2 marks) Now use **Expected Improvement** (simplified):

$$EI(x) = \max(\mu(x) - f^*, 0) + \sigma(x)$$

where $f^* = 0.84$. Compute EI for P, Q, R. Which candidate does EI select? Why are all the max terms equal to 0?

- (c) (2 marks) UCB selects Q, but EI selects R.
- (i) In one sentence: why does EI select R (which has the *lowest* mean) while UCB does not?
- (ii) Find the minimum β for which UCB would also select R over Q. Show the inequality.
- (d) (1 mark) If $\beta = 0$, UCB becomes $\mu(x)$. What strategy is this, and what is its weakness?

Solution

(a) $UCB = \mu + 2\sigma$:

P: $0.81 + 2(0.02) = \mathbf{0.85}$. Q: $0.83 + 2(0.05) = \mathbf{0.93}$. R: $0.75 + 2(0.08) = \mathbf{0.91}$.

UCB selects **Q** (0.93 is highest).

(b) $EI = \max(\mu - 0.84, 0) + \sigma$:

P: $\max(0.81 - 0.84, 0) + 0.02 = 0 + 0.02 = \mathbf{0.02}$.

Q: $\max(0.83 - 0.84, 0) + 0.05 = 0 + 0.05 = \mathbf{0.05}$.

R: $\max(0.75 - 0.84, 0) + 0.08 = 0 + 0.08 = \mathbf{0.08}$.

EI selects **R** (0.08 is highest). All max terms are 0 because every candidate's predicted mean is *below* the best observed accuracy $f^* = 0.84$, so none is expected to improve on the current best based on mean alone.

(c)

(i) EI clips all means below f^* to 0, so R's score depends *only* on its uncertainty $\sigma = 0.08$ (the highest). UCB adds $\beta\sigma$ to μ , so R's low mean (0.75) drags its score below Q's.

(ii) $UCB(R) > UCB(Q) \implies 0.75 + 0.08\beta > 0.83 + 0.05\beta \implies 0.03\beta > 0.08 \implies \beta > 8/3 \approx 2.67$.

(d) Pure **exploitation** (greedy): always pick the highest predicted mean. Weakness: never explores uncertain regions, so it can miss better areas of the search space.

13. (3 points) [Subjective — Dockerfile] A classmate shares a Dockerfile:

```
1 FROM python:3.10
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 EXPOSE 7860
6 CMD ["python", "app.py"]
```

- (a) (1 mark) Suggest **one change** to make the Docker image significantly **smaller**.
- (b) (1 mark) Running `docker run app` starts the container successfully (logs show “Running on 0.0.0.0:7860”), but opening `localhost:7860` in the browser shows nothing. What flag is missing from the `docker run` command?
- (c) (1 mark) The app writes prediction files to `/app/outputs/` inside the container. After running `docker stop`, the files are gone. What Docker feature would preserve them on the host machine? Write the flag you would add to `docker run`.

Solution

- (a) Use `python:3.10-slim` instead of `python:3.10`. The full image is ~ 1 GB; the slim variant is ~ 150 MB — nearly $7\times$ smaller.
- (b) Port mapping: `docker run -p 7860:7860 app`. Without `-p`, the container's port is not exposed to the host.
- (c) A **volume**: `docker run -v $(pwd)/outputs:/app/outputs app`. Mounts a host directory into the container; files persist after the container stops.